Date
Page

Unit 4

DeadLocks

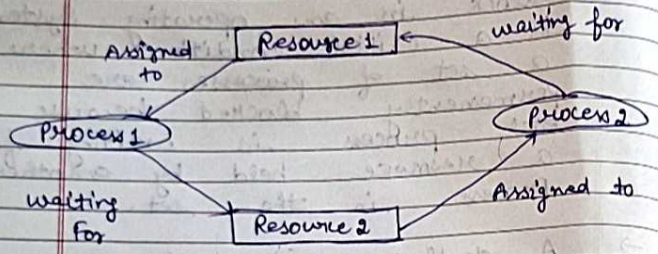
→ Deadlock in an operating system is a critical condition where a set of processes are permanently blocked because each process is waiting for a resource held by another process in the set.

→ A deadlock occurs when:
→ Process P1 is waiting for a resource held by P2
→ Process P2 is waiting for a resource held by P1
→ Neither can proceed → System get stuck
→ In Simple terms:-
"Processes are waiting for each other forever".

Causes of Deadlock in OS:→

→ A process in an OS typically uses resources in the following sequence
(i) Request a Resource
(ii) Use the resource
(iii) Release the resource.

Deadlock arises when processes hold some resources while waiting for others.



Necessary conditions for Deadlock:→

Deadlock occurs only if all 4 conditions hold simultaneously.

1. Mutual Exclusion:→

(i) Resource can be used by only one process at a time.
Eg. Printer.

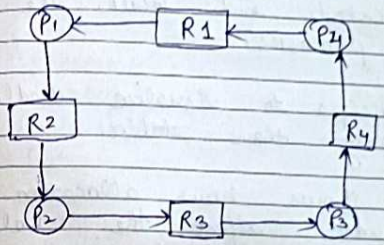
(2) Hold & wait:→

Process holds one resource & waits for another.

(3) No Preemption:→

Resources can not be forcefully preemptive.

(4) Circular Wait:→ Set of processes are waiting for each other in a circular chain.



→ P1 holding R1 and waiting for R2
→ P2 holding R2 and waiting for R3
→ P3 holding R3 and waiting for R4
→ P4 holding R4 and waiting for R1

Resource Allocation Graph (RAG)

→ It is a visual way to understand how resources are assigned in an operating System.

→ Instead of using only tables to show which resources are allocated, requested or available, the RAG uses nodes and edges to clearly illustrate relationship b/w processes and their required resources.

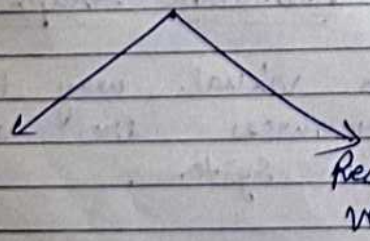
* → RAG shows which resources are allocated and which are requested by processes.

→ It helps to visualize deadlocks more clearly than tables.

→ RAG shows how allocation and requests affect the whole system.

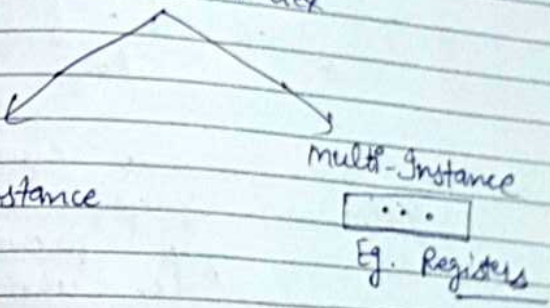
→ Process → circles
Resources → squares
Edges → allocation or Request.

Types of vertices in RAG:→



(1) Process vertex:→ Every process will be represented as a process vertex & represented with a circle.

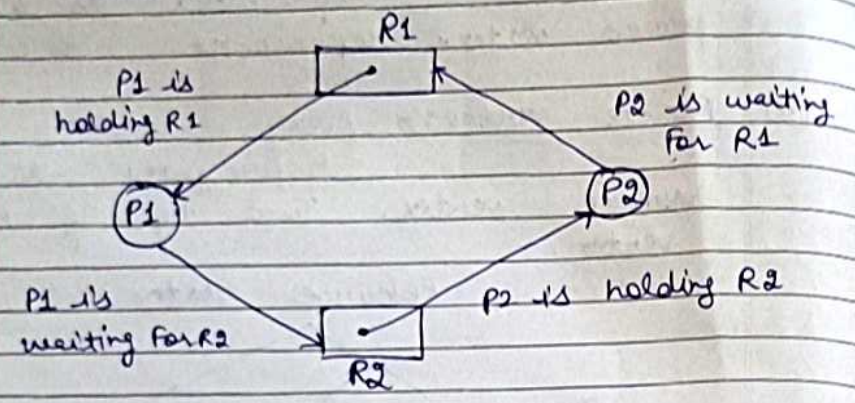
(2) Resource vertex:→ Every resource will be represented as a resource vertex. Two types of resource vertex.



(i) Single Instance:→ A resource with only one copy. In RAG, it is shown as a single node and only one process can use it at a time.

(ii) multi-Instance Resource:→ A resource with multiple copies. In RAG, it is shown with multiple instances, so several processes can use different copies simultaneously.

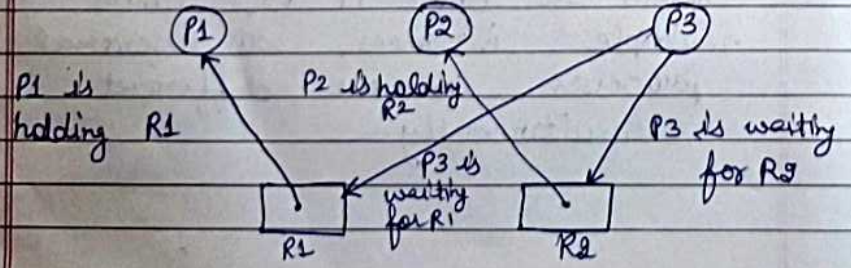
Eg: Single Instance RAG:- (with deadlock)



→ If there is a cycle in Resource allocation graph and each resource in the cycle provides only one instance, then the processes will be in deadlock.

→ For eg, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.

Eg: Single Instance Resource without Deadlock

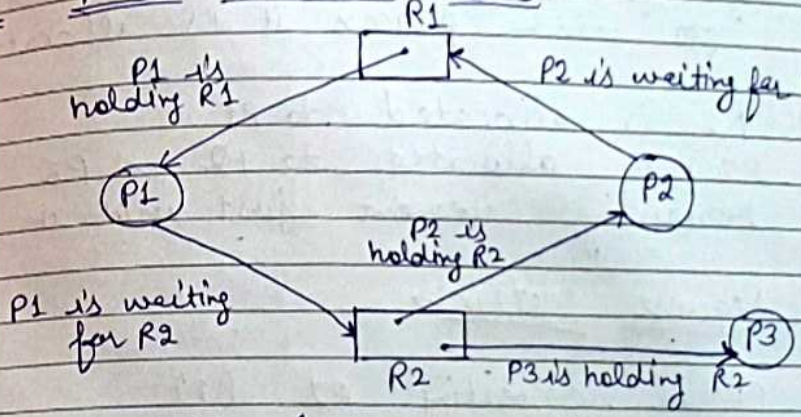


→ Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources.

→ In this example, there is no deadlock because ~~because~~ there is no circular dependency. (Chain)

→ So cycle in single-instance resource type is the sufficient condition for deadlock.

Eg: 2 Multi-Instances RAG:-



Multi-Instances without Deadlock

→ from above example, it is not possible to say the RAG is in a safe state or in a unsafe state.

Process	Allocation Resource		Request Resource	
	R_1	R_2	R_1	R_2
P_1	1	0	0	1
P_2	0	1	1	0
P_3	0	1	0	0

→ Total number of processes → P_1, P_2 & P_3
→ Total no. of resources → R_1 & R_2

→ Allocation matrix → For constructing the allocation matrix, just go to the resources and see to which process it is allocated.

→ R_1 is allocated to P_1
→ R_2 is allocated to P_2 & P_3
→ Remaining element just write 0.

⇒ Request matrix →

→ P_1 is requesting R_2 (1)
→ P_2 is requesting R_1 (1)
→ Remaining element write 0.

⇒ Available resource is (0, 0)

→ Checking deadlock: (Safe or not)

→ Available = [0 0] ✓
P_3 0 1

→ As P_3 does not require any extra resource to complete the execution & after completion P_3 release its own resource.

→ New Available = 0 1
P_1 1 0

→ As using new available resource we can satisfy the requirement of process P_1 (1 0) and P_1 also release its previous resource.

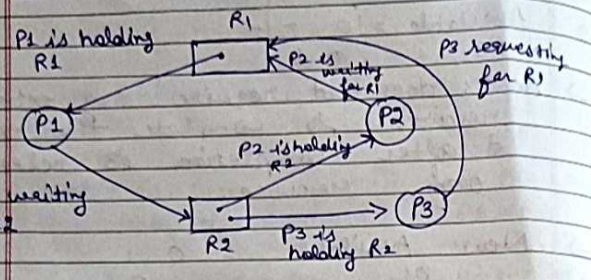
→ New Available = 1 1
P_2 0 1

→ Now easily we can satisfy the requirement of process P_2 (0 1)

→ New Available = 1 2

→ So there is no deadlock in the RAG. Even though there is a cycle, still there is no deadlock.

Multi Instances with Deadlock :->



Process	Allocation Resource		Request Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	1	0

Deadlock Handling Techniques :->

(a) Deadlock Prevention
(ii) Deadlock Avoidance
(iii) Deadlock Detection
(b) Deadlock Recovery

(1) Deadlock Prevention :->

Deadlock prevention is a technique in which the OS make sure that deadlock never happen by denying at least one of the necessary condition of deadlock.

Necessary conditions of deadlock
(i) mutual Exclusion
(ii) Hold & wait
(iii) No preemption
(iv) Circular wait

If any of the four situation get false, the deadlock will be prevented.

(2) Deadlock Avoidance :->

Deadlock avoidance in operating System is a resource allocation strategy in which the system dynamically check each resource

request and grants it only if the system remains in a safe state.

→ Before allocating resources, the system
(i) Predicts the future state
(ii) checks for a safe sequence of execution.
(iii) If safe → request is granted
(iv) If unsafe → request is denied.

Banker's Algorithm:→

Banker's Algorithm is a deadlock avoidance algorithm used in OS that ensures a system never enters an unsafe state by checking resource allocation before granting it.

→ Why is it called 'Banker's?
Just like bank
→ A banker gives loans carefully
→ Ensures all customers can repay

Similarly → OS gives resources only if system remains safe.

Banker's Algorithm prevents deadlock by avoiding unsafe resource allocation.

Eg: Considering a system with 7 processes and three resources A, B & C.
Resource type A has 13 instances, B has 8 instance and type C has 10 instances

Processes	Allocation			Max Need			Current Available			Max Need - Allocation Remaining		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	1	2	1	8	3	3	3	2	2	7	1	2
P1	2	0	0	3	2	2	5	2	2	1	2	2
P2	2	1	2	6	2	3	7	3	4	4	1	2
P3	1	0	1	9	2	3	8	3	6	8	1	2
P4	1	0	2	4	3	3	9	5	7	3	3	1
P5	1	2	1	5	3	2	11	5	8	4	1	1
P6	2	0	1	7	2	2	12	7	9	5	2	1
Total	10	6	8				13	8	10			

A = 13 − 10 = 3
B = 8 − 6 = 2
C = 10 − 8 = 2
} ∴ remaining available

Step 1:
P0 will not execute

Step 2:
Safe State = P1 → P2 → P4 → P5 → P6 → P0 → P3

Deadlock Detection & Recovery:->

-> Deadlock detection & recovery is the mechanism of detecting & resolving deadlocks in an operating System.

-> In operating systems, deadlock recovery is important to keep everything running smoothly.